claude-windows / 90c0b6b8-7430-4870-98cb-e487eacfa1d7

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\90c0b6b8-7430-4870-98cb-e487eacfa1d7\subagents\workflows\wf_ad5139d3-ec0\agent-a77ddb24637e5cfa6.jsonl`
- SHA-256: `56eda7f77448c9fccd2706084eeb28be0889318f855b5e0ba465943abc6953b5`
- Source modified: `2026-07-04T01:49:54+00:00`
- Imported at: `2026-07-05T16:48:25+00:00`
- project: `wf_ad5139d3-ec0`
- session_id: `90c0b6b8-7430-4870-98cb-e487eacfa1d7`

Transcript

1. user (2026-07-04T01:48:32.873Z)

```
Repo root: C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer. Read
backend/orchestration.py (focus on _maybe_dispatch_remote and _execute_compile) and
backend/api/job_worker.py (_drain_due_jobs, recover_stale_jobs). Follow imports as needed.
CRITICAL ? answer each with exact file:line evidence:
1. What conditions trigger remote L4 dispatch in _maybe_dispatch_remote? Which env vars are
required/read (CLOUD_RUN_JOB, CLOUD_RUN_REGION, GOOGLE_CLOUD_PROJECT, DEPLOYMENT_PROFILE,
others)?
2. Does remote dispatch REQUIRE a gs:// input path? Trace EXACTLY what happens if the input
video is a LOCAL filesystem path (e.g. an uploaded file under /tmp): does it raise, silently
fall back to local execution, or dispatch a job that then can't read the file? Quote the
branching logic.
3. Does the code stage/copy a local input file UP to gs:// (storage.input_root) before
dispatching the L4 job? If yes, give file:line. If NO, state clearly that browser-uploaded local
files are NOT auto-staged (this is the suspected A1 gap).
4. How does compile execute ? in-process on the control-plane service (the #460 drain loop /
recover_stale_jobs)? Explain why --no-cpu-throttling --min-instances 1 is required for it to
make progress.
Return structured findings + gaps.
```

2. assistant (2026-07-04T01:48:34.864Z)

```
I'll analyze the orchestration and job worker code. Let me start by reading the key files.
```

3. user (2026-07-04T01:48:36.698Z)

```
1       """Async job worker ? drain/wake loop (DEFERRED_HARDENING_PLAN #16, EXECUTION_POLICY.md
P3).
2
3       Extracted verbatim from backend.main to shrink the orchestrator; behavior is unchanged.
4       The worker DRAINS the jobs table via `claim_due_job` and honours WAIT_AND_RESUME: a job
5       that raises `JobWaiting` is parked (status 'waiting' + `next_poll_at` + progress) and
6       re-claimed when due ? with no central scheduler (the DB is the clock).
7
8       The monkeypatch contract these functions rely on (and that tests depend on):
9       every cross-function seam ? `db_instance`, `global_config`, `_execute_processing`,
10      `_arm_wake_timer`, `_get_executor`, `_MAX_DRAIN_WAKE_SEC` ? is resolved THROUGH the
11      `backend.main` module object at call-time (`import backend.main as _main`), never as a
12      bare local. `backend.main` re-exports these names, so `monkeypatch.setattr(backend.main,
13      ...)` still intercepts the inter-function calls (e.g. a parked job arms the *patched*
14      `_arm_wake_timer`, so no real daemon timer leaks in tests). Importing `backend.main`
lazily
15      inside the functions also avoids the circular import at module load (main.py imports
this
16      module; this module must not import main at import time).
```

```
    """

    import json
    import logging
    import threading
    import traceback
    from concurrent.futures import ThreadPoolExecutor
    from typing import Any, Dict, Optional

    logger = logging.getLogger(__name__)

    # Async job executor (DEFERRED_HARDENING_PLAN #16). max_workers=1 on purpose: a single
    # self-hosted box serializes GPU work, and the Gemini provider is rate-fragile under
    # parallel uploads (2026-06-10 ops learning: serial uploads + backoff). The segmenters
    # already parallelize their AI handoff internally via their own pools.
    #
    # ? O2 RISK (CODE_CLEANUP_AUDIT §3c): a single worker means ONE stuck or hung job
    # (e.g. hung Gemini generate, hung WSL GPU call, hung Cloud Run poll) blocks ALL
    # subsequent jobs ? they stay 'queued' forever. Mitigations already in place:
    #   - ExecutionPolicy op_timeout_sec kills hung generate/processing calls
    #   - detector timeout_sec kills hung WSL/GPU calls
    #   - Cloud Run poll has a max_wait_sec bound
    #   - ffmpeg/ffprobe subprocesses use bounded media timeouts
    # A future slice should add a per-job wall-clock timeout (the executor itself
    # cannot enforce timeouts on the Thread).
    _job_executor: Optional[ThreadPoolExecutor] = None


    def _get_executor() -> ThreadPoolExecutor:
        """Lazy module-global executor; tests monkeypatch this for inline execution."""
        global _job_executor
        if _job_executor is None:
            _job_executor = ThreadPoolExecutor(
                max_workers=1, thread_name_prefix="jobworker"
            )
        return _job_executor


    class JobWaiting(Exception):
        """Raised inside the pipeline to SELF-SCHEDULE a job (EXECUTION_POLICY.md P3,
        WAIT_AND_RESUME). It means "I can't finish now ? park me and resume in `delay_sec`",
        NOT a failure. The worker persists `next_poll_at` + `progress` (via
`set_job_waiting`),
        releases the thread, and re-claims the job when due (`claim_due_job`). No work to
date
        is the segmenter's `_execute_processing`, run UNDER the destroy-after-confirm
contract.

        The wiring is additive: the production segmenter path never raises this today (zero
        behaviour change), so a job runs exactly as before. The hook is what a later slice
        (resumable AI handoff) raises once a long op exhausts its in-process wait budget.
        """

        def __init__(self, delay_sec: float, progress: Optional[str] = None) -> None:
            super().__init__(f"job parked for {delay_sec:.0f}s")
            self.delay_sec = max(0.0, float(delay_sec))
            self.progress = progress


    def _job_to_request(job: Dict[str, Any]):
        """Reconstruct the submit-time ProcessRequest from a claimed job row, so a job
        re-claimed by `claim_due_job` (queued OR resumed-from-waiting) runs identically to
        the original submit. The row stores exactly the ProcessRequest fields."""
        import backend.main as _main

```

```
        raw_params = job.get("params")
        params = (
            raw_params if isinstance(raw_params, dict) else json.loads(raw_params or "{}")
        )
        return _main.ProcessRequest(
            video_path=job["video_path"],
            player_pool=job.get("player_pool"),
            max_frames=job.get("max_frames"),
            segmenter=job.get("segmenter"),
            # v8 compute-picker: the worker runs _execute_processing ?
_maybe_dispatch_remote, which
            # honours this, so it MUST survive submit?reconstruct (unlike locale, which is
only read off
            # the job row for analytics). NULL ? server default = pre-picker behaviour.
            compute=job.get("compute"),
            force=bool(params.get("force")),
        )


    def _run_claimed_job(job: Dict[str, Any], config: Any, db: Any) -> None:
        """Run ONE job already claimed (`status='running'`) by `claim_due_job`, recording
its
        terminal state ? OR parking it for resume on `JobWaiting` (WAIT_AND_RESUME).

        Job `status` is the EXECUTION state (queued|waiting|running|done|failed): 'done'
means
        the worker finished ? the pipeline's own verdict (validation failure etc.) lives in
        result["status"]; 'failed' means the worker itself crashed/raised; 'waiting' means
the
        job self-scheduled and will be re-claimed when `next_poll_at` is due.
        """
        import backend.main as _main

        job_id = job["id"]
        # Dispatch by kind: 'compile' (#329, the async reel stitch) vs 'process'
(NULL/default ? the
        # original /api/process pipeline). Both record their terminal state the same way;
only process
        # jobs feed the M8 cost ledger + M11 flywheel (a compile has no detection
usage/segments).
        is_compile = job.get("kind") == "compile"
        try:
            if is_compile:
                result = _main._execute_compile(
                    config,
                    db,
                    job,
                    progress=lambda stage: db.set_job_progress(job_id, stage),
                )
            else:
                req = _main._job_to_request(job)
                result = _main._execute_processing(
                    config,
                    db,
                    req,
                    progress=lambda stage: db.set_job_progress(job_id, stage),
                )
            if result.get("video_id"):
                db.set_job_video_id(job_id, result["video_id"])
            db.mark_job_done(job_id, result)
            if not is_compile:
                _maybe_record_job_cost(
                    job, result, config, db
                )  # M8 cost ledger (default-OFF; process jobs only)
                _maybe_run_flywheel(
```

```
                    job, result, config
                )  # M11 data-flywheel (process jobs only)
            except _main.JobWaiting as w:
                # Cooperative re-enqueue: persist progress + next_poll_at, release the worker.
                logger.info(f"Job {job_id} parked for {w.delay_sec:.0f}s (resume when due).")
                db.set_job_waiting(job_id, w.delay_sec, progress=w.progress)
            except Exception as e:
                logger.error(f"Job {job_id} crashed: {e}\n{traceback.format_exc()}")
                db.mark_job_failed(job_id, f"{type(e).__name__}: {e}")


    def _maybe_record_job_cost(
        job: Dict[str, Any], result: Dict[str, Any], config: Any, db: Any
    ) -> None:
        """M8 cost ledger (COMPUTE_DECOUPLED_SERVING M8, D8). DEFAULT-OFF: a no-op unless
        ``config.billing.enabled``. When on, compute the job's cost from ``result['usage']``
        (= {gpu_seconds, wall_seconds, egress_gb, gemini_usd, gpu_type}) × the versioned
pricebook, and
        persist it.

        ? **Honest gap (M8 scope):** the pipeline does **not** emit ``result['usage']`` yet,
so even with
        ``billing.enabled=True`` AND a real calibrated pricebook, every recorded cost is
**0** until a
        FOLLOW-UP wires the worker to emit usage (gpu/wall/egress/gemini). M8 lands the
ledger + pricebook
        + the recording seam; the usage-emission step is separate (the metering hooks on the
real run).
        With the placeholder pricebook the cost is 0 regardless.

        Best-effort ? never fails a completed job because cost bookkeeping hiccuped. ``job``
is the
        claimed row (carries ``owner_id``); ``result`` is the pipeline output."""

        billing_cfg = getattr(config, "billing", None)
        if not billing_cfg or not getattr(billing_cfg, "enabled", False):
            return
        try:
            usage = (result or {}).get("usage") or {}
            db.record_job_cost(
                job["id"],
                usage=usage,
                pricebook_version=billing_cfg.pricebook_version,
                gpu_type=usage.get("gpu_type"),
                compute_backend=(result or {}).get("compute_backend"),
                owner_id=job.get("owner_id"),
                margin=billing_cfg.margin,
            )
            logger.info(
                "Recorded cost for job %s (pricebook=%s).",
                job["id"],
                billing_cfg.pricebook_version,
            )
        except Exception as e:  # never wedge a completed job on bookkeeping
            logger.warning(
                "Cost recording for job %s failed (non-fatal): %s", job.get("id"), e
            )


    def _maybe_run_flywheel(
        job: Dict[str, Any], result: Dict[str, Any], config: Any
    ) -> None:
        """M11 data-flywheel (COMPUTE_DECOUPLED_SERVING M11, D12). DEFAULT-OFF + INERT: a
no-op unless
        ``config.flywheel.enabled`` AND attested consent ? and consent is a **faked hard-
```

```
deny** today
194         (``flywheel.consent_attested`` returns False for every job until counsel + the
consent schema
195         land), so this captures **nothing** in production. When activated it captures the
consented job's
196         artifacts into the per-video workspace (? later labeling ? the golden training set).
197
198         Best-effort ? never fails a completed job because the flywheel hiccuped. ``job`` is
the claimed
199         row (carries ``owner_id``); ``result`` is the pipeline output."""
200
201         flywheel_cfg = getattr(config, "flywheel", None)
202         if not flywheel_cfg or not getattr(flywheel_cfg, "enabled", False):
203             return
204         try:
205             from backend import flywheel
206
207             flywheel.run_for_job(job, result, config)
208         except Exception as e:  # never wedge a completed job on the flywheel
209             logger.warning("Flywheel for job %s failed (non-fatal): %s", job.get("id"), e)
210
211
212     def _drain_due_jobs(config: Any, db: Any) -> int:
213         """One worker tick: claim and run every currently-runnable job (queued, or waiting
whose
214         `next_poll_at` is due), earliest-due first, until the table has nothing runnable.
215
216         Each parked job (`JobWaiting` ? status 'waiting') drops out of this drain and is
picked
217         up by a FUTURE drain once due; we schedule that follow-up drain via
`_schedule_drain` so
218         a single-worker box keeps making progress with no central timer (the DB is the
clock).
219         Returns the number of jobs that reached a terminal/parked state this tick.
220         """
221         import backend.main as _main
222
223         ran = 0
224         while True:
225             job = db.claim_due_job()
226             if job is None:
227                 break
228             ran += 1
229             _main._run_claimed_job(job, config, db)
230         # If anything is parked-but-not-yet-due, arrange a wake-up so it resumes without a
new
231         # submit. On the single-box executor this is a delayed re-drain; multi-worker just
relies
232         # on the next submit/drain. Best-effort ? never raises into the worker.
233         _main._schedule_next_drain_if_waiting(config, db)
234         return ran
235
236
237     def _schedule_next_drain_if_waiting(config: Any, db: Any) -> None:
238         """If any job is parked in 'waiting', arm a re-drain for when the soonest one is
due, so a
239         self-scheduled job resumes with no further client submit. The drain itself re-checks
240         due-ness, so an early wake is harmless and a clamped wake for a far-future job is
fine."""
241         import backend.main as _main
242
243         try:
244             delay = db.seconds_until_next_due()
245         except Exception as e:  # pragma: no cover - defensive; never wedge the worker
246             logger.error(f"Could not compute next-due delay: {e}")
```

```
247             return
248         if delay is None:
249             return  # nothing waiting
250         # Clamp so a far-future job doesn't pin a thread forever; the drain re-checks due-
ness.
251         _main._arm_wake_timer(max(0.0, min(delay, _main._MAX_DRAIN_WAKE_SEC)), config, db)
252
253
254     def _arm_wake_timer(delay_sec: float, config: Any, db: Any) -> None:
255         """Arm a single daemon timer that re-submits a drain after ``delay_sec`` (no central
256         scheduler ? the DB is the clock). Isolated as the sole timer-creating seam so tests
can
257         monkeypatch it to a no-op and assert the *scheduling decision* without real
waiting."""
258         import backend.main as _main
259
260         timer = threading.Timer(
261             delay_sec,
262             lambda: _main._get_executor().submit(_main._drain_due_jobs, config, db),
263         )
264         timer.daemon = True
265         timer.start()
266
267
268     #: A parked job further out than this is re-checked by a capped wake rather than one
long
269     #: sleeping timer ? keeps the wake cadence bounded without a central scheduler.
270     _MAX_DRAIN_WAKE_SEC = 60.0
271
```

4. user (2026-07-04T01:48:36.722Z)

```
1       """Processing / compile orchestration, extracted from ``backend.main`` (P23; re-cut of
#390).
2
3       The full hash -> validate -> segment -> report pipeline (``_execute_processing``), the
4       Cloud Run dispatch + ingest path (``_maybe_dispatch_remote`` /
``_ingest_remote_segments``),
5       and the shared compile resolution + worker handler (``_resolve_compile`` /
``_execute_compile``).
6       Extracted verbatim from ``backend.main`` to shrink the orchestrator; behavior is
unchanged.
7
8       SEAM DISCIPLINE (same convention as ``backend.api.job_worker``): anything that must
remain
9       monkeypatchable-on-``backend.main`` or that lives in ``backend.main`` (the transitional
10      ``db_instance``/``global_config`` globals and the ``ProcessRequest`` model) is resolved
through
11      the ``backend.main`` module object at call time (``import backend.main as _main``),
never as a
12      bare local. ``backend.main`` re-exports every name below, so
``monkeypatch.setattr(backend.main,
13      ...)`` and ``import backend.main as m; m._execute_processing(...)`` keep working
unchanged.
14      This module must NOT import ``backend.main`` at module level (main imports us for the
re-export).
15      Routed seams (rebind-patched on ``backend.main`` by tests): ``global_config``,
``db_instance``,
16      ``ProcessRequest``, ``compute_video_id``, ``emit_indexer_report``, ``VideoCompiler``.
17      Object-attribute patches (``m.storage.fetch``, ``m.segmenter_registry.get``, ...) hit
the shared
18      objects and need no routing.
19      """
20
21      import csv
```

```
    import json
    import logging
    import math
    import os
    import tempfile
    from typing import TYPE_CHECKING, Any, Dict, List, Optional, Tuple
    from urllib.parse import quote

    from backend import storage
    from backend.config import AppConfig, StitchingConfig
    from backend.pipeline.segmenters.base import segmenter_registry
    from backend.pipeline.stitcher.watermark_i18n import localize_watermark
    from backend.pipeline.validators.base import registry
    from backend.results import Failure
    from backend.utils.paths import output_base, safe_output_filename

    if (
        TYPE_CHECKING
    ):  # runtime import would be circular (main imports us for the re-export)
        from backend.main import ProcessRequest  # noqa: F401

    logger = logging.getLogger(__name__)


    def _finalize_reingest(db, video_id: str, segments, play_wm: int, cand_wm: int) -> None:
        """Commit a (re)segmentation with destroy-AFTER-confirm semantics.

        On a non-empty run, drop the PRIOR rows (id <= the pre-run watermark) so only the
        fresh segments remain. On an empty run, drop the NEW partial rows (id > watermark)
and
        keep the prior good results intact ? a failed/empty re-run never destroys prior
data.
        """
        if segments:
            db.prune_segments(video_id, play_upto=play_wm, cand_upto=cand_wm)
        else:
            db.prune_segments(video_id, play_after=play_wm, cand_after=cand_wm)


    def _validation_failure_message(failures: List[Any]) -> str:
        details = []
        for failure in failures:
            name = getattr(failure, "validator_name", None) or "validation"
            message = (getattr(failure, "message", None) or "").strip()
            details.append(f"{name}: {message}" if message else str(name))
        if not details:
            return "Video failed validation checks."
        return "Video failed validation: " + "; ".join(details)


    def _report_config_for_input(config: Any, raw_input: str) -> Any:
        """Return report config that keeps remote-input reports out of disposable
scratch."""
        storage_config = (
            config.get("storage")
            if isinstance(config, dict)
            else getattr(config, "storage", None)
        )
        try:
            if storage.input_is_local(raw_input, storage_config):
                return config
        except ValueError:
            return config
        try:
            cfg = (
```

```
                config.model_dump(mode="json")
                if hasattr(config, "model_dump")
                else dict(config or {})
            )
        except Exception:
            cfg = {}
        report = dict(cfg.get("report") or {})
        if report.get("output_dir"):
            return config
        report["output_dir"] = os.path.join(output_base(config), "reports")
        cfg["report"] = report
        return cfg


    def _ingest_remote_segments(
        db, video_id: str, outputs_uri: str, storage_config: dict, *, max_rows: int = 20000
    ) -> int:
        """Read a cloud job's ``intervals.csv`` (start,end,shot_count,ending_reason) from
the bucket and
        persist each rally via the SAME ``db.add_play_segment`` the in-process segmenters
call ? so
        ``/api/query`` + ``/api/compile`` read the rows with zero knowledge of where they
were produced.
        Returns the count ingested. Mirrors ``cloud_run._read_json``'s fetch-to-a-real-dst
pattern (cloud
        backends REQUIRE a dst). Hardened against a hostile/corrupt worker output: a
malformed row is
        skipped, non-finite / overflowing values are rejected, the loop is bounded, and the
temp dir is
        cleaned. May raise (missing/unreadable CSV, fetch error) ? the caller treats that as
a job failure
        and prunes any partial rows (destroy-AFTER-confirm)."""
        uri = f"{outputs_uri.rstrip('/')}/intervals.csv"
        ref = storage.parse_uri(uri)
        n = 0
        with tempfile.TemporaryDirectory(prefix="cr-ingest-") as td:
            local = storage.fetch(
                uri, os.path.join(td, ref.name or "intervals.csv"), config=storage_config
            )
            with open(local, newline="", encoding="utf-8") as f:
                for row in csv.DictReader(f):
                    if n >= max_rows:
                        logger.warning(
                            "[API] cloud ingest hit the %d-row cap (%s) ? ignoring the
rest.",
                            max_rows,
                            uri,
                        )
                        break
                    try:
                        start, end = float(row["start"]), float(row["end"])
                    except (KeyError, TypeError, ValueError):
                        continue
                    if not (math.isfinite(start) and math.isfinite(end)):
                        continue  # reject inf/nan times rather than persist a junk rally
                    meta: Dict[str, Any] = {
                        "source": "cloud_run",
                        "ending_reason": (row.get("ending_reason") or "").strip(),
                    }
                    sc = (row.get("shot_count") or "").strip()
                    if sc:
                        try:
                            meta["shot_count"] = int(
                                float(sc)
                            )  # OverflowError on 'inf'/'1e400'
```

```
                        except (ValueError, OverflowError):
                            meta["shot_count"] = sc
                    db.add_play_segment(video_id, start, end, metadata=meta)
                    n += 1
        return n


    def _cloud_available(cfg) -> bool:
        """Whether THIS server can satisfy a per-request ``compute="cloud"`` dispatch (item
3).

        True when the control plane is wired for Cloud Run: either the configured default
target is
        already ``cloud_run`` (the operator chose cloud serving), or the Cloud Run env
coordinates
        (``CLOUD_RUN_JOB`` + ``GOOGLE_CLOUD_PROJECT``) and an output bucket
(``storage.output_root``)
        are present so a forced per-request override can actually dispatch + collect a
result.

        Surfaced in ``/api/capabilities`` so the SPA only offers the cloud toggle when it
would work."""
        deployment = getattr(cfg, "deployment", None)
        if (getattr(deployment, "compute_target", "") or "") == "cloud_run":
            return True
        has_job = bool(
            os.environ.get("CLOUD_RUN_JOB") and os.environ.get("GOOGLE_CLOUD_PROJECT")
        )
        out_root = (getattr(getattr(cfg, "storage", None), "output_root", "") or "").strip()
        return bool(has_job and out_root)


    def _maybe_dispatch_remote(
        cfg,
        db,
        req: "ProcessRequest",
        video_id: str,
        seg_name: str,
        val_results,
        play_wm: int,
        cand_wm: int,
        notify,
    ) -> Optional[Tuple[Dict[str, Any], bool]]:
        """Cloud-serving (COMPUTE_DECOUPLED_SERVING M6 ? the API path): when
``deployment.compute_target
        == "cloud_run"``, run the heavy DETECT on a Cloud Run GPU Job and INGEST its rally
intervals into
        THIS DB, so the rest of the flow (query ? compile/local-stitch) is unchanged.

        Returns ``(response_dict, ran)`` when the cloud path is taken ? ``ran`` is whether
the cloud job
        actually EXECUTED (``False`` for a pre-dispatch config rejection, so the
observability report
        doesn't claim a fake segmentation run) ? or ``None`` to fall through to the in-
process segmenter
        (**DEFAULT-OFF**: ``get_compute_backend`` returns ``None`` for every
non-``cloud_run`` target, so
        the in-process path stays byte-identical)."""
        from backend.compute import ComputeJobSpec, get_compute_backend

        def _failed(msg: str, ran: bool) -> Tuple[Dict[str, Any], bool]:
            # Shape-match the in-process segmenter-fail branch + drop any partial NEW rows
(id > play_wm);
            # prior good rows (id <= play_wm) are preserved (destroy-AFTER-confirm).
            db.prune_segments(video_id, play_after=play_wm, cand_after=cand_wm)
```

```
193             # Provenance even on failure, and HONEST about what ran: `path` is "cloud_run"
only when the
194             # job actually dispatched + executed remotely (ran=True, e.g. a non-zero worker
rc / ingest
195             # error); a pre-dispatch reject (ran=False) ran NOWHERE ? "not_run". `target`
records the
196             # intended backend, `requested` the picker choice, `dispatched` whether the
remote job ran.
197             return (
198                 {
199                     "video_id": video_id,
200                     "status": "failed",
201                     "message": msg,
202                     "results": [r.model_dump() for r in val_results],
203                     "play_segments": [],
204                     "compute": {
205                         "path": "cloud_run" if ran else "not_run",
206                         "requested": req.compute,
207                         "target": "cloud_run",
208                         "dispatched": ran,
209                     },
210                 },
211                 ran,
212             )
213
214         # SPA compute-picker (item 3): a per-request choice overrides the server's
configured default.
215         # "local" forces the in-process segmenter; "cloud" forces a Cloud Run dispatch
(guarded by
216         # _cloud_available so we fail clearly rather than silently running local); None /
unknown ? default.
217         choice = (req.compute or "").strip().lower()
218         if choice and choice not in ("local", "cloud"):
219             logger.warning(
220                 "[API] ignoring unknown compute=%r (expected 'local'|'cloud'); using server
default.",
221                 req.compute,
222             )
223             choice = ""
224         if choice == "local":
225             return None  # explicit "run on my machine" ? in-process regardless of the
server's target
226         if choice == "cloud":
227             if not _cloud_available(cfg):
228                 return _failed(
229                     "cloud-serving was requested but this server isn't configured for it "
230                     "(no Cloud Run target). Pick 'run on my machine', or configure cloud
serving.",
231                     False,
232                 )
233             compute = get_compute_backend(cfg, target_override="cloud_run")
234         else:
235             compute = get_compute_backend(
236                 cfg
237             )  # server default (default-OFF unless cloud_run)
238         if compute is None:
239             return None  # default-OFF ? run the in-process segmenter below (unchanged)
240
241         storage_cfg = getattr(cfg, "storage", None)
242
243         # The Cloud Run job fetches the input FROM THE BUCKET ? it can't read a path local
to this box.
244         try:
245             input_ref = storage.resolve_input_ref(req.video_path, storage_cfg)
246         except ValueError as e:
```

```
            return _failed(
                f"cloud-serving: unresolvable input '{req.video_path}': {e}", False
            )
        if input_ref.backend == "local":
            return _failed(
                "cloud-serving needs a cloud input (e.g. gs://?); a path local to this
machine "
                "can't be read by the Cloud Run job. Put the video in your bucket first.",
                False,
            )
        out_root = (getattr(storage_cfg, "output_root", "") or "").strip()
        if not out_root:
            return _failed(
                "cloud-serving needs storage.output_root set to a bucket (e.g. gs://your-
bucket) "
                "? that's where the Cloud Run job writes the result.",
                False,
            )

        notify("dispatching (cloud_run)")
        logger.info(
            "[API] cloud-serving: dispatching %s -> Cloud Run Job (out=%s)",
            req.video_path,
            out_root,
        )
        # Forward the engine's AI-handoff stance so the cloud job runs the SAME detection:
with
        # skip_ai_handoff=True the WASB windows become rallies directly (no Gemini key
needed); without
        # forwarding the cloud job would default to the handoff and yield 0 rallies when it
has no key.
        # (player_pool / max_frames are still dropped on the cloud path ? a separate
documented follow-up.)
        overrides = []
        sk = getattr(getattr(cfg, "indexing", None), "skip_ai_handoff", None)
        if sk is not None:
            overrides.append(f"indexing.skip_ai_handoff={'true' if sk else 'false'}")
        spec = ComputeJobSpec(
            video_uri=req.video_path,
            out_uri=out_root,
            segmenter=seg_name,
            config_overrides=tuple(overrides),
        )
        try:
            result = compute.run_infer(
                spec
            )  # dispatch the Cloud Run Job + bucket-poll to completion
        except Exception as e:  # noqa: BLE001 ? any dispatch/poll error is a clean job
failure, not a 500
            logger.error("[API] cloud dispatch error: %s", e, exc_info=True)
            return _failed(f"cloud dispatch error: {e}", True)
        if result.returncode != 0:
            return _failed(f"cloud job failed (returncode {result.returncode}).", True)
        if result.video_id and result.video_id != video_id:
            # The worker re-hashes the bucket bytes; a divergence means the input changed
between this
            # box's hash and the worker's ? ingest under the LOCAL id but flag it so it's
diagnosable.
            logger.warning(
                "[API] cloud video_id %s != local %s ? input bytes differ between the API
hash "
                "and the worker hash; ingesting under the local id.",
                result.video_id,
                video_id,
            )
```

```
302
303         notify("ingesting (cloud_run)")
304         storage_dict = (
305             storage_cfg.model_dump()  # type: ignore[union-attr]  # hasattr-guarded; mypy
can't narrow
306             if hasattr(storage_cfg, "model_dump")
307             else {}
308         )
309         try:
310             _ingest_remote_segments(db, video_id, result.outputs_uri, storage_dict)
311         except Exception as e:  # noqa: BLE001 ? a missing/partial/corrupt result must not
500 or leak rows
312             logger.error("[API] cloud ingest failed: %s", e, exc_info=True)
313             return _failed(
314                 f"cloud-serving: could not read the result from {result.outputs_uri}: {e}",
315                 True,
316             )
317
318         # Only the NEW cloud rows (id > the pre-run watermark); _finalize_reingest then
drops the old ones.
319         segments = [
320             s for s in db.query_play_segments(video_id, {}) if int(s.get("id", 0)) > play_wm
321         ]
322         _finalize_reingest(db, video_id, segments, play_wm, cand_wm)
323         # Compute-path PROVENANCE: the GCP-verifiable proof the cloud path ran. `execution`
is the real
324         # Cloud Run execution name (cross-check: gcloud run jobs executions describe
<execution>).
325         provenance = {
326             "path": "cloud_run",
327             "requested": req.compute,
328             "job": os.environ.get("CLOUD_RUN_JOB", "rally-worker"),
329             "region": os.environ.get("CLOUD_RUN_REGION", "asia-south1"),
330             "execution": result.execution,
331             "outputs_uri": result.outputs_uri,
332         }
333         logger.info(
334             "[COMPUTE] cloud_run dispatched: execution=%s job=%s region=%s out=%s
video_id=%s",
335             result.execution,
336             provenance["job"],
337             provenance["region"],
338             result.outputs_uri,
339             video_id,
340         )
341         return (
342             {
343                 "video_id": video_id,
344                 "status": "success",
345                 "message": f"Successfully indexed {len(segments)} play segments (cloud_run /
'{seg_name}').",
346                 "results": [r.model_dump() for r in val_results],
347                 "play_segments": segments,
348                 "compute": provenance,
349             },
350             True,
351         )
352
353
354     def _execute_processing(
355         config=None, db=None, req=None, progress=None
356     ) -> Dict[str, Any]:
357         """The full hash ? validate ? segment ? report pipeline for one video.
358
359         Accepts explicit ``config``/``db`` (the new app-factory pattern) OR reads the
```

```
        module-level ``global_config``/``db_instance`` (backward-compatible with existing
        tests that pass only ``req``). Route handlers always pass explicit params; older
        test code that calls ``_execute_processing(req)`` still works.

        ``progress`` is an optional callable(stage: str) used to surface coarse job
progress.
        Raises on unexpected internal errors ? the caller records those as a job failure
        """
        import backend.main as _main  # call-time seam: main's globals/model stay patchable

        # Backward-compatible shim: if called as _execute_processing(req) (old pattern),
        # shift positional args ? req arrives in position 0 (config), progress in position 1
(db).
        if config is not None and not isinstance(config, AppConfig):
            # Old signature: _execute_processing(req, progress=None)
            if req is None and db is not None and isinstance(db, _main.ProcessRequest):
                req = db
                db = None
            elif req is None and isinstance(config, _main.ProcessRequest):
                req = config
                config = None
        # Resolve config/db from module globals when not passed explicitly
        if config is None:
            config = _main.global_config
        if db is None:
            db = _main.db_instance

        notify = progress or (lambda stage: None)

        notify("hashing")
        # M2: resolve the input to a LOCAL path. A bare/local path is returned unchanged
(identity ?
        # byte-identical to today); a bucket/Drive URI is fetched to scratch first. The
original
        # req.video_path (the source URI) is kept for the DB record + report; only the file-
reading
        # consumers (hash / validators / segmenter) use the resolved local path.
        local_video = ""
        try:
            local_video = storage.acquire_local_input(
                req.video_path, getattr(config, "storage", None)
            )
            video_id = _main.compute_video_id(local_video)
        except Exception:
            storage.cleanup_acquired_local_input(
                req.video_path, local_video, getattr(config, "storage", None)
            )
            raise
        try:
            existing_video = db.get_video(video_id)
        except Exception:
            storage.cleanup_acquired_local_input(
                req.video_path, local_video, getattr(config, "storage", None)
            )
            raise
        was_reingest = existing_video is not None

        # typed AppConfig: attribute access for the default segmenter (with safe fallback)
        default_seg = getattr(
            getattr(config, "indexing", None), "default_segmenter", "motion"
        )
        seg_name = req.segmenter or default_seg
        segmenter_actual = None
        segmentation_ran = False
        val_results: List[Any] = []
```

```
420         val_context: Dict[str, Any] = {}
421         # Compute-path PROVENANCE (the validation method): which backend actually ran the
DETECT ?
422         # "in_process" once the local segmenter is reached, set on the response as
"cloud_run" by
423         # _maybe_dispatch_remote when it dispatches. None ? nothing ran (validation/config
fail).
424         compute_actual: Optional[str] = None
425         response: Optional[Dict[str, Any]] = None
426         try:
427             if (
428                 existing_video
429                 and existing_video.get("status") == "processed"
430                 and not req.force
431             ):
432                 cached_segments = db.query_play_segments(video_id, {})
433                 if cached_segments:
434                     logger.info(
435                         "[API] Reusing cached segmentation for video_id=%s; force=true
bypasses cache.",
436                         video_id,
437                     )
438                     response = {
439                         "video_id": video_id,
440                         "status": "success",
441                         "message": (
442                             f"Reused {len(cached_segments)} cached play segments. "
443                             "Pass force=true to reprocess this video."
444                         ),
445                         "results": existing_video.get("validation_results", []),
446                         "play_segments": cached_segments,
447                         "cached": True,
448                         "compute": {
449                             "path": "cached",
450                             "requested": getattr(req, "compute", None),
451                         },
452                     }
453                     return response
454
455             # 1. Run pluggable validators (with-context variant surfaces ffprobe_meta for
the report)
456             notify("validating")
457             logger.info(f"Running validations for {req.video_path}...")
458             val_results, val_context = registry.run_all_with_context(local_video, config)
459             failures = [r for r in val_results if not r.passed]
460
461             if failures:
462                 # add_video is an UPSERT (no cascade) ? a failed re-validation only updates
463                 # status; it no longer destroys the video's prior good segments.
464                 db.add_video(
465                     video_id,
466                     req.video_path,
467                     "failed",
468                     [r.model_dump() for r in val_results],
469                 )
470                 response = {
471                     "video_id": video_id,
472                     "status": "failed",
473                     "message": _validation_failure_message(failures),
474                     "results": [r.model_dump() for r in val_results],
475                 }
476                 return response
477
478             # 2. Run segmenter & indexer
479             logger.info("[API] Video passed checks. Segmenting and indexing...")
```

```
            db.add_video(
                video_id, req.video_path, "processed", [r.model_dump() for r in val_results]
            )

            segmenter_cls = segmenter_registry.get(seg_name)
            if not segmenter_cls:
                # Submit-time validation already 400s unknown names; this is the defensive
                # in-worker path (e.g. config default pointing at an unregistered
segmenter).
                available = list(segmenter_registry.list_available().keys())
                response = {
                    "video_id": video_id,
                    "status": "failed",
                    "message": f"Segmenter '{seg_name}' not found. Available segmenters:
{available}",
                    "results": [r.model_dump() for r in val_results],
                    "play_segments": [],
                }
                return response

            logger.info(f"[API] Using segmenter: {seg_name}")
            notify(f"segmenting ({seg_name})")
            segmenter_actual = seg_name
            # Destroy-AFTER-confirm: remember the pre-run rows, then keep new + drop old
only
            # if the run yields segments; otherwise drop the new partial rows and keep the
old.
            play_wm, cand_wm = db.segment_watermarks(video_id)
            # COMPUTE_DECOUPLED_SERVING M6 ? API: with deployment.compute_target=cloud_run,
run the heavy
            # DETECT on a Cloud Run GPU Job + ingest its rallies HERE so query/compile are
unchanged.
            # DEFAULT-OFF: returns None for every non-cloud target ? the in-process
segmenter below runs
            # byte-identically. (player_pool/max_frames aren't threaded to the worker
`infer` CLI yet, so
            # they're dropped on the cloud path in this slice ? a documented follow-up.)
            remote = _maybe_dispatch_remote(
                config, db, req, video_id, seg_name, val_results, play_wm, cand_wm, notify
            )
            if remote is not None:
                response, segmentation_ran = (
                    remote  # `ran` = did the cloud job actually execute (report)
                )
                return response  # the cloud branch already stamped response["compute"]
(path=cloud_run)
            # In-process from here on ? record the actual path for the provenance stamp in
`finally`.
            compute_actual = "in_process"
            segmenter = segmenter_cls(db, config)
            try:
                result = segmenter.process_video(
                    video_id, local_video, req.player_pool, req.max_frames
                )
            except Exception:
                # A raising segmenter must not leave half-written rows: restore the pre-run
                # state (same destroy-after-confirm contract as the Failure branch), then
let
                # the job worker record the failure.
                db.prune_segments(video_id, play_after=play_wm, cand_after=cand_wm)
                raise
            segmentation_ran = True

            if isinstance(result, Failure):
                logger.error(f"[API] Segmenter failed: {result.message}")
```

```
                db.prune_segments(video_id, play_after=play_wm, cand_after=cand_wm)
                response = {
                    "video_id": video_id,
                    "status": "failed",
                    "message": f"Segmenter '{seg_name}' failed: {result.message}",
                    "results": [r.model_dump() for r in val_results],
                    "play_segments": [],
                }
                return response

            segments = result.value if hasattr(result, "value") else result
            notify("finalizing")
            _finalize_reingest(db, video_id, segments, play_wm, cand_wm)

            response = {
                "video_id": video_id,
                "status": "success",
                "message": f"Successfully indexed {len(segments)} play segments using
'{seg_name}' segmenter.",  # type: ignore[arg-type]  # hasattr-unwrap above; Success arm has
.value
                "results": [r.model_dump() for r in val_results],
                "play_segments": segments,
            }
            return response
        finally:
            # Compute-path PROVENANCE stamp (the validation method): the cloud branch
already set
            # response["compute"] (path=cloud_run + the Cloud Run execution); for every
other path stamp
            # the actual path here so the job result PROVES which backend ran (in_process /
not_run) and
            # what the picker REQUESTED ? a silent fallback (requested cloud, ran local) is
then visible.
            if isinstance(response, dict):
                response.setdefault(
                    "compute",
                    {
                        "path": compute_actual or "not_run",
                        "requested": getattr(req, "compute", None),
                    },
                )
            prov = response.get("compute", {}) if isinstance(response, dict) else {}
            logger.info(
                "[COMPUTE] video_id=%s requested=%s actual=%s dispatched=%s status=%s",
                video_id,
                getattr(req, "compute", None),
                prov.get("path"),
                prov.get("dispatched"),
                (response or {}).get("status") if isinstance(response, dict) else None,
            )
            # Always emit the per-video observability report (next to the video, or to
            # report.output_dir). Never raises. Surface the paths in the response. (The
compute-path
            # proof lives on the job result + the [COMPUTE] log, not the report ?
obsreport's record_run
            # doesn't surface arbitrary run signals.)
            paths = _main.emit_indexer_report(
                db=db,
                video_id=video_id,
                # Resolved LOCAL path, not the raw URI: obsreport writes the report next to
the
                # video via os.path.abspath(video_path), and a remote URI (gs://, gcs://,
local://)
                # collapses to a literal scheme dir at the repo root (see #397). The source
URI is
```

```
                # still preserved in DB provenance via db.add_video.
                video_path=local_video,
                config=_report_config_for_input(config, req.video_path),
                val_results=val_results,
                val_context=val_context,
                segmenter_requested=req.segmenter,
                segmenter_actual=segmenter_actual,
                was_reingest=was_reingest,
                segmentation_ran=segmentation_ran,
            )
            if paths and isinstance(response, dict):
                response["reports"] = paths
            storage.cleanup_acquired_local_input(
                req.video_path, local_video, getattr(config, "storage", None)
            )


    def _remote_input_size_bytes(input_ref, storage_config) -> Optional[int]:
        """Best-effort size (bytes) of a remote storage source, or ``None`` if unknowable.

        Used only to pre-reserve scratch for a fetch-to-local (P2). Never raises: a stat that needs a
        missing cloud SDK, the network, or a credential returns ``None`` so the free-space guard degrades
        to a no-op rather than blocking a job it cannot measure."""
        try:
            sc = (
                storage_config.model_dump()
                if hasattr(storage_config, "model_dump")
                else (storage_config or {})
            )
            backend = storage.get_storage(input_ref.backend, config=sc)
            return int(backend.stat(input_ref).size)
        except Exception as e:  # noqa: BLE001 - measuring is best-effort; inability must not block
            logger.debug(
                "free-space: could not stat remote source %s: %s", input_ref.uri, e
            )
            return None


    class _CompileError(Exception):
        """A submit-time-validatable compile problem ? carries the HTTP status + detail so the API path
        maps it to an HTTPException (fast 4xx) and the worker path maps it to a failed-job result."""

        def __init__(self, status_code: int, detail: str):
            super().__init__(detail)
            self.status_code = status_code
            self.detail = detail


    def _resolve_compile(
        db, cfg, video_id: str, segment_ids: List[int], output_filename: str
    ):
        """Shared compile resolution ? run at SUBMIT (fast-fail 4xx) AND in the WORKER (re-resolve, since a
        row could change between submit and run). Verifies the video exists, the requested segments match
        and survive the ``stitching.min_shot_count`` floor (segments WITHOUT a known shot_count are exempt
        so manual/legacy picks can't silently vanish), and the output name is traversal-safe. Raises
        ``_CompileError(404/400)``. Returns ``(video_row, kept_segments,
```

```
safe_out_name)``."""
        video = db.get_video(video_id)
        if not video:
            raise _CompileError(404, "Indexed video record not found.")
        # Reject path traversal / absolute names (os.path.join would otherwise write
anywhere on disk).
        try:
            out_name = safe_output_filename(output_filename)
        except ValueError as e:
            raise _CompileError(400, str(e)) from e

        all_segments = db.query_play_segments(video_id, {})
        matched = [s for s in all_segments if s["id"] in segment_ids]
        if not matched:
            raise _CompileError(400, "No matching play segments found to stitch.")

        stitching = getattr(cfg, "stitching", None) or StitchingConfig()
        kept = []
        for s in matched:
            meta = s.get("metadata") or {}
            sc = meta.get("shot_count") if isinstance(meta, dict) else None
            try:
                if sc is not None and int(sc) < stitching.min_shot_count:
                    continue
            except (TypeError, ValueError):
                pass  # unparseable count -> treat as unknown, keep
            kept.append(s)
        if len(kept) < len(matched):
            logger.info(
                "[compile] stitching.min_shot_count=%s: dropped %d/%d segments below the
floor.",
                stitching.min_shot_count,
                len(matched) - len(kept),
                len(matched),
            )
        if not kept:
            raise _CompileError(
                400,
                f"All {len(matched)} matching segments fall below "
                f"stitching.min_shot_count={stitching.min_shot_count}.",
            )
        return video, kept, out_name


    def _execute_compile(cfg, db, job: Dict[str, Any], progress=None) -> Dict[str, Any]:
        """Worker handler for a ``kind='compile'`` job (#329): re-resolve the segments,
stitch the reel,
        and return the {status, filepath, download_url} result the SPA polls for. Returns a
        ``status='failed'`` result for an EXPECTED problem (missing video / no segments / a
stitch error)
        so the SPA always gets a clean verdict instead of a worker-crash error."""
        import backend.main as _main  # call-time seam: VideoCompiler stays patchable on
backend.main

        notify = progress or (lambda stage: None)
        params = json.loads(job.get("params") or "{}")
        video_id = job.get("video_id") or ""
        segment_ids = params.get("segment_ids") or []
        locale = params.get("locale")

        notify("resolving")
        try:
            video, kept, out_name = _resolve_compile(
                db, cfg, video_id, segment_ids, params.get("output_filename", "")
            )
```

```
703         except _CompileError as e:
704             return {"status": "failed", "message": e.detail, "video_id": video_id}
705
706         stitching = getattr(cfg, "stitching", None) or StitchingConfig()
707         output_dir = (
708             getattr(getattr(cfg, "output", None), "output_dir", "output") or "output"
709         )
710         # Localize the branding watermark to the UI locale the SPA rendered in (recorded on
the job);
711         # unknown/absent locale keeps the configured default unchanged.
712         localized_watermark = localize_watermark(stitching.watermark, locale)
713         compiler = _main.VideoCompiler(
714             output_dir,
715             begin_padding=stitching.begin_padding,
716             end_padding=stitching.end_padding,
717             watermark=localized_watermark,
718         )
719         time_pairs = [(s["start_time"], s["end_time"]) for s in kept]
720
721         notify("stitching")
722         local_src = ""
723         try:
724             # The stored `filepath` is the SOURCE ref (gs://, s3://, gdrive://, or a local
path). Resolve to
725             # a LOCAL file the stitcher can read ? identity for a local path, fetch-to-
scratch for a URI (the
726             # same seam the process path uses); without it, compiling a bucket-ingested
video can't open gs://.
727             local_src = storage.acquire_local_input(
728                 video["filepath"], getattr(cfg, "storage", None)
729             )
730             out_path = compiler.compile_highlights(local_src, time_pairs, out_name)
731         except Exception as e:  # noqa: BLE001 ? a stitch/ffmpeg failure is a clean job
FAILURE, not a crash
732             logger.error(
733                 "[compile] stitching failed for video_id=%s: %s", video_id, e, exc_info=True
734             )
735             return {
736                 "status": "failed",
737                 "message": f"Highlight stitching failed: {e}",
738                 "video_id": video_id,
739             }
740         finally:
741             if local_src:
742                 storage.cleanup_acquired_local_input(
743                     video["filepath"], local_src, getattr(cfg, "storage", None)
744                 )
745
746         # `filepath` is server-local (not browser-reachable); also return the download URL
so the SPA needn't
747         # hand-build it (P6 / PER_RALLY_SELECTION.md §4.3).
748         download_url = f"/api/highlights/{quote(video_id)}/{quote(out_name)}"
749         notify("finished")
750         return {
751             "status": "success",
752             "filepath": out_path,
753             "download_url": download_url,
754             "video_id": video_id,
755         }
756
```

5. assistant (2026-07-04T01:48:41.212Z)

Both files are read. Let me follow the key imports to verify the tracing, particularly `storage.resolve_input_ref`, `resolve_input_ref` backend detection, and `acquire_local_input` to

confirm whether local files get staged up.

6. user (2026-07-04T01:48:43.838Z)

```
backend\storage\__init__.py:50:def resolve_input_ref(raw: str, storage_config: Any = None) ->
StorageRef:
backend\storage\__init__.py:66:def input_is_local(raw: str, storage_config: Any = None) -> bool:
backend\storage\__init__.py:74:def acquire_local_input(
backend\storage\ref.py:51:def parse_uri(uri: str) -> StorageRef:
```

7. user (2026-07-04T01:48:44.044Z)

```
backend\storage\base.py
backend\storage\gdrive.py
backend\storage\local.py
backend\storage\ref.py
backend\storage\s3.py
backend\storage\capabilities.py
backend\storage\__init__.py
backend\storage\gcs.py
backend\storage\gdrive_api.py
```

8. user (2026-07-04T01:48:48.022Z)

```
40
41      def _storage_dict(storage_config: Any) -> Dict[str, Any]:
42          """Coerce a StorageConfig (Pydantic) / dict / None into a plain settings dict."""
43          if storage_config is None:
44              return {}
45          if hasattr(storage_config, "model_dump"):
46              return storage_config.model_dump()  # type: ignore[no-any-return]
47          return dict(storage_config)
48
49
50      def resolve_input_ref(raw: str, storage_config: Any = None) -> StorageRef:
51          """Resolve a user-supplied input (a path or storage URI) into a ``StorageRef`` under
the input
52          config (``input_backend``/``input_root``). Raises ``ValueError`` for an unsupported
URI scheme ?
53          callers at the API/CLI boundary catch that and return a clean client error (never a
500).
54
55          Use ``ref.backend`` to branch local-vs-remote and ``ref.key`` for the resolved LOCAL
path (e.g.
56          ``local://?`` is stripped to the bare path), rather than stat-ing the raw URI
string."""
57          sc = _storage_dict(storage_config)
58          uri = resolve_input_uri(
59              raw,
60              input_backend=sc.get("input_backend", "local"),
61              input_root=sc.get("input_root", ""),
62          )
63          return parse_uri(uri)
64
65
66      def input_is_local(raw: str, storage_config: Any = None) -> bool:
67          """True if ``raw`` resolves (under the input config) to a LOCAL filesystem path ?
i.e. a
68          plain on-disk file the caller can stat/validate directly. False for a bucket / Drive
URI
69          that must be fetched first. Raises ``ValueError`` for an unsupported scheme (same as
70          ``resolve_input_ref``)."""
71          return resolve_input_ref(raw, storage_config).backend == "local"
72
```

```
73
74      def acquire_local_input(
75          raw: str, storage_config: Any = None, *, scratch_parent: "str | None" = None
76      ) -> str:
77          """Resolve ``raw`` (a path or storage URI) to a LOCAL file path ready for
processing.
78
79          Local / bare paths are returned UNCHANGED (identity passthrough, no copy ? today's
behaviour
80          byte-for-byte). A bucket / Drive source (``gs://``/``s3://``/``gdrive://``, or a
bare name under a
81          non-local ``input_root``/``input_backend``) is downloaded into a fresh scratch dir
and the local
82          path returned. The caller owns successful remote scratch copies; use
83          ``cleanup_acquired_local_input`` when the consumer has finished reading the returned
path."""
84          sc = _storage_dict(storage_config)
85          ref = resolve_input_ref(raw, sc)
86          if ref.backend == "local":
87              return (
88                  ref.key
89              )  # identity ? no temp, no copy, byte-identical to passing the path directly
90          scratch = tempfile.mkdtemp(prefix="rally-input-", dir=scratch_parent)
91          try:
92              return fetch(ref.uri, os.path.join(scratch, ref.name or "in.mp4"), config=sc)
93          except Exception:
94              shutil.rmtree(scratch, ignore_errors=True)
95              raise
96
97
98      def cleanup_acquired_local_input(
99          raw: str, local_path: str, storage_config: Any = None
100     ) -> None:
101         """Delete a remote scratch dir produced by ``acquire_local_input``.
102
103         Local inputs are identity paths and must never be removed. Remote acquisitions are
downloaded
104         into a fresh ``rally-input-*`` directory; deleting that directory is safe once the
caller has
105         finished hashing/validating/stitching/segmenting the returned file.
106         """
107         try:
108             if input_is_local(raw, storage_config):
109                 return
110         except ValueError:
111             return
112         if not local_path:
113             return
114         scratch = os.path.dirname(os.path.abspath(local_path))
115         if os.path.basename(scratch).startswith("rally-input-") and os.path.isdir(scratch):
116             shutil.rmtree(scratch, ignore_errors=True)
117
118
119     def _backend_class(name: str):
120         """Lazy-resolve a backend class so cloud SDKs aren't imported until needed."""
121         if name == "local":
122             from backend.storage.local import LocalStorage
123
124             return LocalStorage
125         if name == "s3":
126             from backend.storage.s3 import S3Storage
127
128             return S3Storage
129         if name == "gcs":
130             from backend.storage.gcs import GCSStorage
```

```
131
132             return GCSStorage
133         if name == "gdrive":
134             from backend.storage.gdrive import GDriveStorage
135
136             return GDriveStorage
137         if name == "gdrive-api":
138             from backend.storage.gdrive_api import GDriveApiStorage
139
140             return GDriveApiStorage
141         raise ValueError(
142             f"unknown storage backend: {name!r} (expected local|s3|gcs|gdrive|gdrive-api)"
143         )
144
145
146     def get_storage(
147         backend: Optional[str] = None,
148         *,
149         config: Optional[Dict[str, Any]] = None,
150         **overrides,
151     ) -> StorageBackend:
152         """Construct a backend. ``backend`` (or ``config['backend']``, default 'local')
selects
153         the implementation; its per-backend settings come from ``config[backend]`` merged
with
154         ``overrides``. Settings carry NO secrets ? only endpoints/regions/paths/source-
selectors."""
155         cfg = config or {}
156         name = backend or cfg.get("backend", "local")
157         # Per-backend settings live under the backend name; tolerate the hyphen?underscore
form so a
158         # hyphenated backend ("gdrive-api") can read a pydantic field that can't contain a
hyphen
159         # ("gdrive_api"). Hyphen-free names are unaffected (the first lookup wins).
```

9. user (2026-07-04T01:48:48.452Z)

```
1       """Storage value objects + URI parsing for the pluggable storage layer.
2
3       A ``StorageRef`` is a backend-agnostic pointer to one object. URIs:
4
5           <bare path>                 -> local   (today's behaviour; e.g. C:/v/x.mp4,
/data/x.mp4)
6           local://<path>              -> local   (local:///abs or local://rel/p)
7           s3://<bucket>/<key>         -> s3       (AWS + R2 + DO Spaces + MinIO via
endpoint_url)
8           gcs://<bucket>/<key>        -> gcs      (gs:// accepted, normalized to gcs://)
9           gdrive://<path>             -> gdrive   (path under your locally-MOUNTED Google
Drive "My Drive")
10
11      No cloud SDKs are imported here ? this module is pure stdlib so ``import
backend.storage``
12      stays cheap.
13      """
14
15      from __future__ import annotations
16
17      import re
18      from dataclasses import dataclass
19      from datetime import datetime
20      from typing import Optional
21
22      _SCHEME_RE = re.compile(r"^([A-Za-z][A-Za-z0-9+.\-]*)://")
23      _KNOWN = ("local", "s3", "gcs", "gdrive", "gdrive-api")
24
```

```
25
26      @dataclass(frozen=True)
27      class StorageStat:
28          """Lightweight object metadata (uniform across backends)."""
29
30          size: int
31          modified: Optional[datetime] = None
32          etag: Optional[str] = None
33          content_type: Optional[str] = None
34
35
36      @dataclass(frozen=True)
37      class StorageRef:
38          """A backend + a location. ``bucket`` is the S3/GCS bucket; for ``local`` and
``gdrive`` it is
39          empty and ``key`` carries the path (a filesystem path, or a path under the mounted
Drive)."""
40
41          backend: str
42          bucket: str
43          key: str
44          uri: str
45
46          @property
47          def name(self) -> str:
48              return self.key.rstrip("/").rsplit("/", 1)[-1]
49
50
51      def parse_uri(uri: str) -> StorageRef:
52          """Parse a storage URI (or a bare filesystem path) into a ``StorageRef``.
53
54          A string with no ``scheme://`` prefix (including Windows ``C:\\...`` paths, which
55          have no ``//``) is treated as a local filesystem path ? preserving today's
behaviour.
56          """
57          s = str(uri)
58          m = _SCHEME_RE.match(s)
59          if not m:
60              return StorageRef("local", "", s, f"local://{s}")
61          scheme = m.group(1).lower()
62          rest = s[m.end() :]
63          if scheme == "gs":
64              scheme = "gcs"
65          if scheme in ("local", "gdrive", "gdrive-api"):
66              # Path-keyed, no bucket: local FS, a path under the mounted Drive, or a path
under a Drive-API
67              # root folder (gdrive-api resolves the path ? a Drive file id).
68              return StorageRef(scheme, "", rest, s)
69          if scheme in _KNOWN:
70              bucket, _, key = rest.partition("/")
71              return StorageRef(scheme, bucket, key, s)
72          raise ValueError(f"unsupported storage URI scheme: {scheme}:// (in {s!r})")
73
74
75      def resolve_input_uri(
76          raw: str, *, input_backend: str = "local", input_root: str = ""
77      ) -> str:
78          """Resolve a user-supplied input reference into a storage URI
(COMPUTE_DECOUPLED_SERVING M2).
79
80          Precedence (first match wins):
81            1. An explicit ``scheme://`` on ``raw`` (``gs://``, ``s3://``, ``local://``,
``gdrive://``) ? as-is.
82            2. An ABSOLUTE local path (``/data/x.mp4``, ``C:\\?``, ``C:/?``) ? as-is (always
local).
```

```
83            3. A bare/relative name + a non-empty ``input_root`` ? joined under it
84               (``"gs://bucket/videos"`` + ``"x.mp4"`` ? ``"gs://bucket/videos/x.mp4"``).
85            4. A bare/relative name + ``input_backend != "local"`` ? prefixed (``"gcs"`` +
``"bucket/x"`` ?
86               ``"gcs://bucket/x"``).
87            5. Otherwise ? ``raw`` unchanged (a relative/bare LOCAL path ? today's behaviour).
88
89          So the default (``input_backend="local"``, ``input_root=""``) is an identity
passthrough for every
90          path used today, preserving byte-for-byte behaviour.
91          """
92          s = str(raw)
93          if _SCHEME_RE.match(s):  # 1. explicit scheme wins
94              return s
95          # 2. an anchored LOCAL path is always local, never joined. Detect this OS-
INDEPENDENTLY ?
96          #    os.path.isabs is platform- AND Python-version-dependent (on Windows + py3.13 a
POSIX-style
97          #    "/data/x" is NOT absolute). Anchored == a leading "/" or "\\", or a Windows
drive prefix "X:".
98          if s.startswith(("/", "\\")) or (len(s) >= 2 and s[1] == ":" and s[0].isalpha()):
99              return s
100         if input_root:  # 3. join a relative name under the configured root
101             return input_root.rstrip("/") + "/" + s.lstrip("/")
102         if (
103             input_backend and input_backend != "local"
104         ):  # 4. prefix a bucket/key with the declared backend
105             return f"{input_backend}://{s}"
106         return s  # 5. bare/relative local path ? unchanged
107
```

10. assistant (2026-07-04T01:48:52.985Z)

I now have full evidence to trace the local-path branching. Let me verify a couple more things: the `recover_stale_jobs` function (mentioned in the prompt but not in job_worker.py), and confirm no staging-up logic exists in `_maybe_dispatch_remote`.

11. user (2026-07-04T01:48:55.125Z)

```
backend\infrastructure\database_jobs.py:390:        NOTE: ``recover_stale_jobs`` is the
preferred startup call ? it RE-QUEUES stale
backend\infrastructure\database_jobs.py:407:    def recover_stale_jobs(
backend\main.py:1451:    requeued, failed = db_instance.recover_stale_jobs()
```

12. user (2026-07-04T01:48:56.460Z)

```
deploy\cloudrun\README.md:73:  --no-cpu-throttling \        # CPU stays allocated between
requests ? the drain thread finishes
deploy\cloudrun\README.md:74:  --min-instances 1 \          # one always-warm instance drains
reliably (cheap CPU; NOT the D7 GPU pool)
deploy\cloudrun\README.md:78:Without `--no-cpu-throttling`, a `/api/compile` returns 202 and
then **stalls** ? the reel never
deploy\cloudrun\README.md:79:finishes and startup crash-recovery would mark it terminal. `min-
instances 1` does NOT contradict
docs\archives\past_projects\DECOUPLED_COMPUTE\03-PLATFORM-OPTIONS.md:41:| **Google Cloud** | ?
**$300 / 90 days** (the only clean win that *exceeds* the ask) | Cloud Run (container) · Vertex
Custom Training (container/pkg) | L4 on Cloud Run; Vertex Spot | ? **Cloud Run GPU min-
instances=0 ? exact per-invocation GPU-seconds** | GCS native | **Best for the side goal** (per-
video billing = per-invocation bill) + train on Vertex Spot. IAM/quota friction. |
docs\archives\past_projects\DECOUPLED_COMPUTE\03-PLATFORM-OPTIONS.md:59:   metering is native**
? **GCP Cloud Run with an L4 GPU (min-instances=0)** on the $300 trial, *or*
```

13. assistant (2026-07-04T01:48:59.688Z)

Let me read the two key evidence sources: `recover_stale_jobs` and the README explaining the CPU throttling requirement.

14. user (2026-07-04T01:49:01.187Z)

```
380                 return self.get_job(row["id"])
381             except Exception as e:
382                 logger.error(f"Failed to claim a due job: {e}")
383                 return None
384 
385         def fail_stale_jobs(self, reason: str = "interrupted by server restart") -> int:
386             """Crash recovery: any job still queued/running from a previous process is dead
387             (the executor is in-process). Mark failed so clients see a terminal state, not a
388             hang. Returns the number of jobs swept.
389 
390             NOTE: ``recover_stale_jobs`` is the preferred startup call ? it RE-QUEUES stale
391             ``compile`` jobs (idempotent CPU stitch, safe to retry) instead of failing them.
This
392             blanket-fail method stays for callers/tests that want the old behaviour."""
393             try:
394                 with self._connect() as conn:
395                     cur = conn.cursor()
396                     cur.execute(
397                         "UPDATE jobs SET status='failed', error=?,
finished_at=CURRENT_TIMESTAMP "
398                         "WHERE status IN ('queued', 'running')",
399                         (reason,),
400                     )
401                     conn.commit()
402                     return cur.rowcount
403             except Exception as e:
404                 logger.error(f"Failed to sweep stale jobs: {e}")
405                 return 0
406 
407         def recover_stale_jobs(
408             self, reason: str = "interrupted by server restart"
409         ) -> Tuple[int, int]:
410             """Crash recovery, kind-aware (#329 serving reliability). Called once at
startup.
411 
412             A ``compile`` job is a **CPU-only, idempotent** ffmpeg re-stitch (re-resolving
the same
413             segments yields the same reel), so a compile left queued/running by a killed
instance is
414             **re-queued** ? the startup drain re-runs it and the client keeps polling until
it gets
415             the reel, instead of seeing a spurious "failed". A ``process`` job is an
expensive,
416             non-idempotent GPU detection pass; auto-retrying could double-spend, so it is
**failed**
417             (terminal, as before) and the client re-submits deliberately.
418 
419             Returns ``(requeued_compile, failed_process)``."""
420             try:
421                 with self._connect() as conn:
422                     cur = conn.cursor()
423                     # Re-queue interrupted COMPILE jobs (idempotent ? safe to retry
automatically).
424                     cur.execute(
425                         "UPDATE jobs SET status='queued', started_at=NULL,
next_poll_at=NULL, "
426                         "error=NULL WHERE status IN ('queued', 'running') AND
kind='compile'"
427                     )
428                     requeued = cur.rowcount
```

```
429                     # Fail interrupted PROCESS jobs (GPU, not safe to auto-retry).
430                     cur.execute(
431                         "UPDATE jobs SET status='failed', error=?,
finished_at=CURRENT_TIMESTAMP "
432                         "WHERE status IN ('queued', 'running') AND (kind IS NULL OR kind !=
'compile')",
433                         (reason,),
434                     )
435                     failed = cur.rowcount
436                     conn.commit()
437                     return requeued, failed
438             except Exception as e:
439                 logger.error(f"Failed to recover stale jobs: {e}")
440                 return 0, 0
441
442
```

15. user (2026-07-04T01:49:02.202Z)

```
55      # A `worker infer` (or the API job path) with a gs:// video now DISPATCHES to the Job
instead of
56      # running in-process; the container runs the SAME worker INLINE (compute_target forced
local_gpu).
57      python -m backend.worker infer --video-uri gs://<bucket>/videos/clip.mp4 --out-uri
gs://<bucket> --segmenter wasb_hybrid
58      ```
59      **The AI handoff needs its key IN THE JOB env** (`GEMINI_API_KEY` ? mount a Secret
Manager secret
60      on the job), or run fully-local with `--set indexing.skip_ai_handoff=true` (candidate
windows
61      emitted as rallies ? the eval-parity mode). With neither, the segmenter used to bail to
a silent
62      0-rally "success" **after** the paid GPU pass; the cloud-serving startup checks now fail
this at
63      boot (`AI_HANDOFF_KEY_MISSING`, rc 2) before the video is even pulled.
64
65      ### 3a. Control-plane SERVICE deploy ? required flags for reliable async jobs
66
67      The API server (`/api/process`, `/api/compile` ? #329) runs its **job worker IN-
PROCESS**: the
68      request returns `202` and a background thread drains the job (GPU detection dispatches
to the Job;
69      the reel stitch runs on the control plane itself). On Cloud Run that background thread
only makes
70      progress while the instance has CPU, so a scale-to-zero **service** deploy MUST set:
71      ```bash
72      gcloud run deploy rally-control-plane --image <image> --region $REGION \
73        --no-cpu-throttling \        # CPU stays allocated between requests ? the drain thread
finishes
74        --min-instances 1 \          # one always-warm instance drains reliably (cheap CPU;
NOT the D7 GPU pool)
75        --cpu 2 --memory 4Gi --port 8080 --allow-unauthenticated \
76        --set-env-vars DEPLOYMENT_PROFILE=cloud-serving,CLOUD_RUN_JOB=rally-
worker,CLOUD_RUN_REGION=$REGION,GOOGLE_CLOUD_PROJECT=$PROJECT
77      ```
78      Without `--no-cpu-throttling`, a `/api/compile` returns 202 and then **stalls** ? the
reel never
79      finishes and startup crash-recovery would mark it terminal. `min-instances 1` does NOT
contradict
80      D7 (that was the ~$500/mo warm **GPU** pool); this is a ~1-vCPU CPU instance. Defense-
in-depth is in
81      the code: `Database.recover_stale_jobs()` **re-queues** a compile interrupted by a
restart (idempotent
82      stitch) rather than failing it, so even an unlucky scale-down mid-stitch self-heals on
the next drain.
```

```
83      Reserve `--allow-unauthenticated` for the pre-CF-auth smoke test only; lock ingress to
the CF proxy
84      (M9) before inviting testers.
85
86      ## 4. M7 acceptance (the owner runlog)
87      Capture a `DEPLOY_REPORT_cloud-serving_<date>.md` under
88      `docs/COMPUTE_DECOUPLED_SERVING/runlogs/` (mirrors the truly-local template): the exact
commands,
89      rally counts, `gpu_active_seconds`/`wall_seconds`/`bytes_out`, a reel smoke-test
(`ffmpeg -f null -`
90      exit 0 + a 5-frame contact sheet), gotchas, and the **$0 teardown audit**.
91
92      ## Gotchas
93      - The dispatched container **must run inline** ? `CloudRunCompute` forces
`compute_target=local_gpu`
94        + `indexing.detector_impl=native` in the per-execution args so the Job never re-
dispatches itself.
```

16. user (2026-07-04T01:49:03.073Z)

```
1435            _instance_lock = None
1436        else:
1437            if _instance_lock is None:
1438                print(
1439                    f"[FATAL] Another instance is already using this database ({db_path}).
Refusing to "
1440                    f"start ? a second instance would corrupt in-flight job state. Stop the
other one, or "
1441                    f"use a different --output-dir (RALLY_APP_HOME) to run an independent
instance."
1442                )
1443                sys.exit(1)
1444
1445        db_instance = Database(db_path)
1446
1447        # Crash recovery (#16, #329): jobs left queued/running by a previous process are
dead ?
1448        # the executor is in-process. Compile jobs (idempotent CPU stitch) are RE-QUEUED so
a reel
1449        # interrupted by a Cloud Run scale-down finishes on the next drain instead of
failing the
1450        # client; process jobs (GPU, non-idempotent) are failed so pollers see a terminal
state.
1451        requeued, failed = db_instance.recover_stale_jobs()
1452        if requeued or failed:
1453            logger.warning(
1454                f"[JOBS] Recovered stale jobs from a previous run ? {requeued} compile re-
queued, "
1455                f"{failed} process failed."
1456            )
1457
1458        # O1/P11: sweep Gemini File-API uploads orphaned by a hard process death (the per-
call
1459        # _safe_delete covers normal/except paths, but a SIGKILL/OOM mid-upload leaves a
clip on
1460        # Google's servers forever). Best-effort, video-only, grace-gated; never blocks
startup.
1461        _maybe_sweep_gemini_orphans(global_config)
1462
1463        # CLI processing mode (no web UI needed). --app/--server always run the server,
never CLI single-shot.
1464        if args.video_path and not args.server and not args.app:
1465            print(f"[CLI] Processing video file: {args.video_path}")
1466            # M2: bare/local path = identity (today's behaviour); a bucket/Drive URI is
fetched to scratch.
```

```
1467            # The local-existence fast-fail only applies to a local input (stat the RESOLVED
key, not the
1468            # raw URI); a remote fetch fails loudly. An unsupported scheme is a clean error,
not a crash.
1469            storage_cfg = getattr(global_config, "storage", None)
1470            try:
1471                input_ref = storage.resolve_input_ref(args.video_path, storage_cfg)
1472            except ValueError as e:
1473                print(f"[FAIL] {e}")
1474                return
1475            if input_ref.backend == "local" and not os.path.exists(input_ref.key):
1476                print(f"[FAIL] Video file not found: {args.video_path}")
1477                return
1478            local_video = storage.acquire_local_input(args.video_path, storage_cfg)
1479            try:
```

17. assistant (2026-07-04T01:49:11.614Z)

I have all the evidence I need. Let me confirm there is no upload-staging in `_maybe_dispatch_remote` by checking the whole function has no `put`/`upload`/`store` call to the bucket for the input ? which I can already confirm from the read: after the `input_ref.backend == "local"` check it goes straight to `_failed(...)`. Let me verify the `#460` drain reference and confirm compile runs in-process (not dispatched remotely) ? which is already clear from `_execute_compile` calling `VideoCompiler` locally. I have enough to answer.

18. user (2026-07-04T01:49:54.737Z)

Structured output provided successfully